

座位安排题解

不妨设列数 m 比行数 n 少

动态规划以每一行作为阶段。每一行的状态都可以唯一对应一个 2 进制数（该处放人为 1，不放为 0），一行

也就是有 2^m 种状态。

记 $recWays[j][Stat]$ 为 当前摆放第 i 行 已经摆放了 j 个人 且该行的摆放情况为 $Stat$ 时的方案个数

如果上一行的一种摆放情况 $PrevStat$ 与 $Stat$ 不矛盾. 那么 $recWays[i-1][j-(Stat \text{ 摆放的人个数})][PrevStat]$ 就可以

加到 $recWays[j][Stat]$ 中去....

Doit 所做的是寻找对于某一个 $[j][Stat]$.. 有 哪一些 $[i+1][j'][Stat']$ 和他不矛盾 他就把自己的方案数贡献给他。

标程：

```
#include <cstdio>
#include <cstring>
const int  limitSize = 80 + 10;
const int  limitPick = 20 + 10;
const int  limitStat = 1 << 8;
int  n , m , cntPick;
void init()
{
    scanf("%d%d%d" , &n , &m , &cntPick);
    if( n < m )
    {
        int t = n;
        n = m; m = t;
    } // 交换 使得 m <= n
    }
    __int64  combine [limitSize][limitPick];
    __int64  recWays [limitSize][limitPick][limitStat];
    inline bool testBits( int stat , int bits )
    {
        return (stat & (1 << bits)) ? 1 : 0;
    }
    void doIt( int r , int c , int numPick , int statA , int statB , __int64 numWays )
    {
        if( c >= m )
        {
            recWays[r][numPick][statB] += numWays; return ;
        }
        doIt( r , c + 1 , numPick , statA , statB , numWays );
        if( testBits( statA , c ) || (c && testBits( statB , c - 1 )) )
            return ;
```

```

doIt( r , c + 1 , numPick + 1 , statA , statB + (1 << c) , numWays );
}
__int64 gcd( __int64 a , __int64 b )
{
    if( b == 0 ) return a;
    return gcd( b , a % b );
}
void solve()
{
    int i , k , stat;
    int top = 1 << m;
    combine[0][0] = 1;
    for(i = 1; i <= n * m; i ++)
    {
        combine[0] = 1;
        for(k = 1; k <= i && k <= cntPick; k ++)
            combine[k] = combine[i - 1][k] + combine[i - 1][k - 1];
    } //计算 C(a,b)
    recWays[0][0][0] = 1;
    for(i = 0; i < n; i ++)
        for(k = 0; k <= cntPick; k ++)
            for(stat = 0; stat < top; stat ++)
            {
                if( recWays[k][stat] == 0 ) continue;
                doIt( i + 1 , 0 , k , stat , 0 , recWays[k][stat] );
            }
    __int64 a = 0 , b = combine[n * m][cntPick] , z;
    for(stat = 0; stat < top; stat ++)
        a += recWays[n][cntPick][stat];
    if( a == 0 )
        puts("Impossible!");
    else
    {
        z = gcd( a , b );
        printf("%I64d/%I64d\n" , b / z , a / z);
    }
}
int main()
{
    init();
    solve();
    return 0;
}

```

期望算法:

$$p = 1-q$$

$$\text{Ans} = q(1 + 2p + 3p^2 + \dots)$$

$$S_n = (1 + 2p + 3p^2 + \dots np^{n-1}) = (1-p^n)/((1-p)^2) + np^n/(1-p)$$

$$S(\text{infinity}) = (1 + 2p + 3p^2 + \dots) = 1/(1-p)^2$$

$$\text{Ans} = S(\text{infinity}) * q = 1/(q^2) * q = 1/q$$